United International University

Dept. of Computer Science and Engineering (CSE)

Final Exam Year: 2025 Semester: Summer

Course Code: CSE 2217 Title: Data Structure and Algorithms II

Marks: 40 Time: 2 Hours

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

Answer all the questions. All questions are of values indicated on the right-hand margin.

1

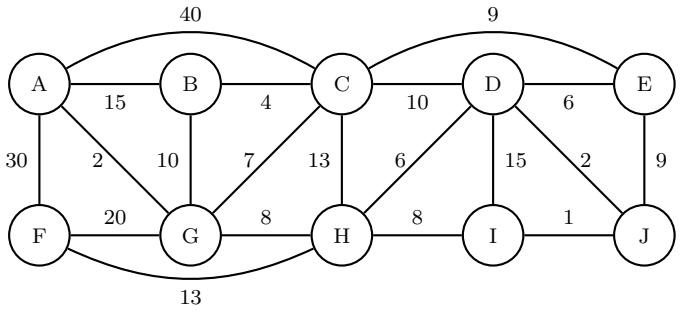


Figure 1a: Question 1(a)

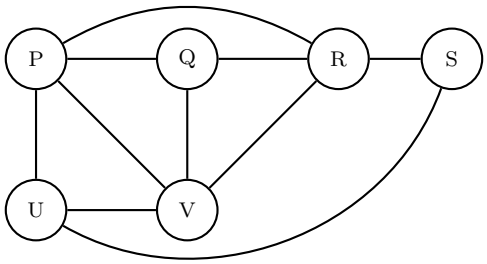


Figure 1b: Question 1(b)

(a) Find the MST for the given graph in Figure 1a using **Prim's algorithm**. Start from node C. Determine the **total cost of the MST**. **[4]**

(b) Consider the graph shown in Figure 1b. Assign weights to each of the edges such that the MST of this graph consists exactly of the edges (S, U), (P, R), (R, V), (U, V), (R, Q). **[2]**

(c) What will be the runtime complexity of Kruskal's algorithm on a graph with $|V|$ vertices and $|E|$ edges if we use bubble sort to sort edges (Sorting n elements with bubble sort takes time $O(n^2)$)? **[1]**

2

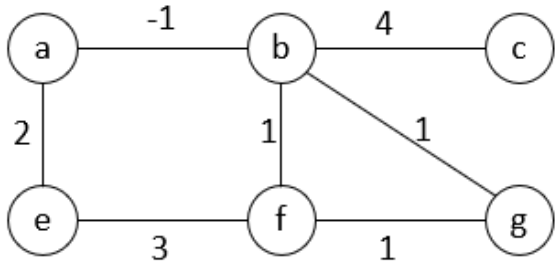


Figure 2a: Graph for Question 2(a)

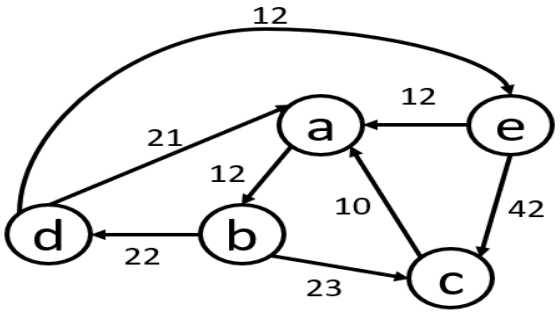


Figure 2b: Graph for Question 2(c)

(a) Consider the graph given in Figure 2a. Is it possible to determine the shortest path from the **vertex a** to **vertex g**? Give reason in favor of your answer. **[3]**

(b) "Dijkstra's algorithm may or may not work on a graph with negative weighted edges" – do you agree with the statement? Give reason in favor of your answer. **[3]**

	(c) Using **Bellman Ford's** algorithm, determine the shortest possible paths from the **vertex c** to all the other vertices in the graph given in Figure 2b. You must show all the necessary calculations and write down the paths along with the distances.	**[4]**
3	**(i)** Why is the **union-by-rank** heuristic used in Disjoint Set Union? Explain how this optimization helps to make the `Union` and `Find-Set` operations faster.	**[3]**
	(ii) The current state of the Disjoint Set Union (DSU) is represented by the following arrays. Perform all the operations below sequentially using the **union-by-rank** and **path compression** heuristics. Give the updated `Parent` and any `Rank` entry after every operation.	

Index	0	1	2	3	4	5	6	7	8	9	10	11
Parent	3	0	1	3	3	4	7	8	8	8	4	9
Rank	0	0	0	3	0	0	0	0	2	0	0	0

	(a) Draw the corresponding disjoint-set forest.	**[1]**
	(b) What is returned by `Find-Set`(2) and by `Find-Set`(11)?	**[1]**
	(c) Perform `Union`(6, 4).	**[1.5]**
	(d) Next perform `Union`(5, 9). If two elements are already in the same set, briefly justify what your union does.	**[1.5]**
4	(a) Consider you are doing double hashing for open-addressing with the following functions for a hash table of size = 7. *h1(k) = (sum of ASCII values of characters in k) mod 7* *h2(k) = 5 - ((sum of ASCII values of characters in k) mod 5)* Final probe sequence: *h(k,i) = (h1(k) + i * h2(k)) mod 7, for i = 0,1,2,...* **Note:** Use ASCII values: a=97, b=98, c=99, d=100, ...	
	i. Find the initial hash values for 'ab', 'bc', 'cd' and 'ba'.	**[2]**
	ii. Insert them (from i.) in the hash table in order and then 'da'. Show the simulation for each entry.	**[3]**
	iii. Delete 'ab' from the table and then search for 'ba'. What will happen in this scenario and what can be done to resolve this case?	**[1]**
	(b) Consider you are doing hashing with (X mod 17) where X is the input value. What will happen if the size of the array is more than 17 or less than 17? Explain with examples for both cases.	**[2]**

5 (a) You are given a string `S = "nemoney"` and a pattern `P = "one"`. **[5]**
The alphabet set is **{e, m, n, o, y}**.

You must find all occurrences of the pattern within the string using the **Rabin–Karp string-matching algorithm**.

Use the following **hash function** and **rolling-hash technique**:

$$h(s) = \left(s[0] \times d^{n-1} + s[1] \times d^{n-2} + \cdots + s[n-1] \times d^{0}\right) \bmod q$$

Where:
d = number of unique characters in the alphabet
n = length of the substring
$q = 59$

Each character is represented by the following integer codes:

Character	e	m	n	y	o
Code	3	5	7	2	11

Show the indices of both **valid hits** and **spurious hits** (if any) using the aforementioned hash function with detailed step-by-step simulation.

(b) You have to perform a DNA test by comparing a sample DNA pattern with a long DNA sequence. It is less likely that the DNA sequence contains many repeated occurrences of the pattern. Between the **Rabin–Karp** algorithm and the **Naive** string-matching algorithm, which one would you choose? Justify your answer with **mathematical reasoning**. **[2]**